PROJECT BASED LEARNING REPORT

CLOUD TASK PLACEMENT USING HYBRID ALGORITHMS

Design, Implementation and Performance Analysis of a Hybrid Algorithmic System for Latency-Aware and Capacity-Constrained Task Placement in the Cloud

Course Name: Design Analysis And Algorithm

Semester: 03

Academic Year: 2026–27

Submitted by

S. No.	Name	Register Number
1	**Andrew Devasani**	**25WU0102067**
2	**Sathyanarayana Varma**	**25WU0102088**
3	**G. Daiwik Reddy**	**25WU0102340**
4	**G. Keerthan Chowdary**	**25WU0102076**

Woxsen University

School of Technology

B.Tech – Computer Science and Engineering

Under the Guidance of

Faculty Name: Dr. Syed Javeed Pasha

Department of Computer Science and Engineering

October 2026

CERTIFICATE

This is to certify that the project titled **"Cloud Task Placement Using Hybrid Algorithms"** is a bonafide work carried out by the following students of B.Tech – Computer Science and Engineering during the Academic Year 2026–27, under my supervision and guidance.

S. No.	Name	Register Number
1	Andrew Devasani	25WU0102067
2	Sathyanarayana Varma	25WU0102088
3	G. Daiwik Reddy	25WU0102340
4	G. Keerthan Chowdary	25WU0102076

The project has been completed as part of the requirements of the Project Based Learning component of the B.Tech – Computer Science and Engineering programme.

Guide Signature: _____

Name of Guide: Dr. Syed Javeed Pasha

Department: Computer Science and Engineering

DECLARATION

We hereby declare that the project titled **"Cloud Task Placement Using Hybrid Algorithms"** is our original work carried out under the guidance and supervision of the faculty member mentioned in this report.

The project was developed as part of the academic requirements of the B.Tech – Computer Science and Engineering programme at Woxsen University. The design, implementation, testing, experimentation, analysis, and documentation presented in this report represent the collective work of the project team.

We confirm that this work has not been submitted, either wholly or partly, for the award of any degree, diploma, certificate, or other academic qualification at this or any other institution.

Wherever external books, research papers, technical documentation, algorithms, or software frameworks have been referred to, appropriate acknowledgement has been provided through numbered citations and references.

We further declare that the information presented in this report accurately represents the project developed and evaluated by the team.

Place: Hyderabad

Date: _____

Signatures

S. No.	Name	Register Number	Signature
1	Andrew Devasani	25WU0102067	
2	Sathyanarayana Varma	25WU0102088	
3	G. Daiwik Reddy	25WU0102340	
4	G. Keerthan Chowdary	25WU0102076	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Woxsen University, the School of Technology, and the Department of Computer Science and Engineering for providing us with the opportunity to undertake this Project Based Learning project.

We are thankful to our faculty guide, **Dr. Syed Javeed Pasha**, for his guidance, suggestions, technical feedback, and encouragement throughout the project. His feedback helped us understand how theoretical concepts from Design and Analysis of Algorithms can be applied to a practical problem.

We would also like to acknowledge the academic resources and research publications that helped us understand spanning trees, shortest-path algorithms, greedy techniques, dynamic programming, and assignment problems [1]–[10].

This project provided us with an opportunity to work as a team and understand the importance of dividing a complex problem into smaller modules. Through implementation and testing, we gained practical experience in algorithm design, software development, debugging, experimentation, and performance analysis.

Finally, we thank everyone who contributed directly or indirectly to the successful completion of this project.

Project Team

Andrew Devasani

Sathyanarayana Varma

G. Daiwik Reddy

G. Keerthan Chowdary

ABSTRACT

Cloud task placement is the problem of deciding which server runs each task. Efficient planning becomes difficult when servers have limited capacity and tasks have different demands, values, and user locations.

This project develops a hybrid algorithmic system for placing tasks under these constraints. The cloud network is represented as a weighted graph. A **minimum spanning tree** (Kruskal and Prim) gives a low-cost backbone [3], [4], and **Dijkstra's algorithm**, Bellman-Ford, Floyd-Warshall and Johnson compute latencies [5]–[9]. A **greedy proposal** step lets every task ask for its best server, and **0/1 Knapsack Dynamic Programming** selects the most valuable subset of tasks on each server within its capacity [10]. The system is compared with nearest-first and random placement, with a greedy variant, and with an exact integer-programming optimum.

The system is implemented in Python. On synthetic data, Johnson's algorithm was about **12 times faster** than Floyd-Warshall at 200 servers. The placement reached about **95% of the optimal utility** on small instances and kept average latency about **3.5 times lower** than nearest-first under overload. The MST backbone cut link cost by **82-84%** while lowering placement utility by only **0.7-1.2%**. The project also includes controlled experiments and **7 automated tests**, all passing.

Keywords: Cloud task placement, Minimum spanning tree, Dijkstra, Johnson, Knapsack, Dynamic Programming, Greedy, Assignment problem.

TABLE OF CONTENTS

S. No.	Chapter / Section	Page No.
1	Certificate	i
2	Declaration	ii
3	Acknowledgement	iii
4	Abstract	iv
5	Table of Contents	v
6	List of Tables	vi
7	List of Figures	viii
8	Chapter 1 – Introduction	10
9	Chapter 2 – Problem Statement and Objectives	11
10	Chapter 3 – Literature Review	14
11	Chapter 4 – Methodology and System Design	16
12	Chapter 5 – Implementation	24
13	Chapter 6 – Results and Discussion	27
14	Chapter 7 – Conclusion and Future Scope	36
15	Chapter 8 – Team Contributions	38
16	References	38
17	Appendix	39

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1.1	Key Terms	11
Table 2.1	Project Objectives	13
Table 3.1	Sources and Their Role in the Project	14
Table 3.2	Knapsack and Cloud Placement Mapping	15
Table 4.1	Sample Servers	17
Table 4.2	Sample Graph	17
Table 4.3	Sample Tasks	18
Table 4.4	Dataset Parameters	18
Table 4.5	Algorithms and Their Time Complexity	23
Table 5.1	Technologies Used	24
Table 5.2	Implementation Notes	25
Table 5.3	Automated Tests	26
Table 6.1	Kruskal's Algorithm on the Sample Network	27
Table 6.2	Dijkstra's Algorithm from Server 0	27
Table 6.3	Latency Matrix L(a, b) in Milliseconds	28
Table 6.4	Optimized Placement Result	28
Table 6.5	Proposal Rounds on the Sample Network	29
Table 6.6	Knapsack Table for Server 0, Round 1	29
Table 6.7	Performance on the Sample Network	30
Table 6.8	Algorithm Comparison on the Sample Network	30
Table 6.9	Runtime Against Network Size (ms)	30
Table 6.10	All-Pairs Runtime on 120 Servers (ms)	31
Table 6.11	Quality Against the Exact Optimum	32
Table 6.12	Results on 60 Servers and 250 Tasks (Average of 4 Seeds)	33
Table 6.13	MST Backbone Against the Full Network	33
Table 6.14	Placement with Full-Network Latencies Against Tree-Only Latencies	34
Table 6.15	Dynamic Programming Against a Greedy Selection Rule	34
Table 6.16	Results for Different Values of λ (60 Servers, 250 Tasks, Load 1.0)	35
Table 6.17	Rounds Used by the Placement Algorithm (Average of 4 Seeds)	35
Table 6.18	Placement Time at Load 0.9	36

Table No.	Table Title	Page No.
Table 6.19	Summary of Experimental Findings	36
Table 8.1	Team Contributions	38
Table A.1	Sample Input Data – Network Links	39
Table A.2	Sample Input Data – Servers	39
Table A.3	Sample Input Data – Tasks	40
Table A.4	Sample Output	40
Table A.5	Project Technologies	41

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 4.1	Proposed System Architecture	16
Figure 4.2	Sample Six-Server Network (orange links form the MST)	17
Figure 4.3	Placement Algorithm Workflow	22
Figure 5.1	Placement Output on a 40-Server Network	25
Figure 6.1	Runtime Against Network Size (Log Scale)	31
Figure 6.2	All-Pairs Runtime Against Number of Links (120 Servers)	31
Figure 6.3	Utility as a Percentage of the Exact Optimum (12 Small Instances)	32
Figure 6.4	Total Utility and Average Latency as Load Increases	32
Figure 6.5	Effect of the Latency Penalty on Average Latency and Tasks Placed	35

CHAPTER 1 — INTRODUCTION

1.1 Background

A cloud is a collection of servers, usually grouped in data centres, connected by a network. Customers do not buy individual machines. They submit tasks such as virtual machines or containers, and the provider decides which physical server runs each one. This decision is called placement.

Placement matters for three reasons. First, every server has limited capacity (CPU cores, memory), so a task placed on a full server cannot run. Second, the network delay between the task and its users decides how fast it responds, and delay grows with the distance and the number of links in between. Third, tasks differ in how valuable they are to the provider, for example because of the price paid or the priority of the customer.

The network also has a cost. Every link between data centres has to be built, rented and maintained, so the provider wants the network to be as cheap as possible while still connecting every server. This is a different question from where to run tasks, but the two interact: the network decides how far apart servers are, and distance decides the latency that placement tries to minimise.

This project models the problem using a weighted graph and combines several classical algorithms [1], [2] to obtain a feasible and valuable placement plan.

1.2 Motivation

The project was developed to demonstrate how Design and Analysis of Algorithms (DAA) concepts can be applied to a practical cloud computing problem.

The major algorithms have different roles:

- **Kruskal / Prim:** cheapest backbone network
- **Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson:** latency between servers
- **Greedy proposals:** each task asks for its best server
- **Knapsack DP:** best subset of tasks on each server
- **Exact solver:** optimum on small instances, used as a yardstick
- **Baselines:** nearest-first and random placement

1.3 Scope

The system supports:

- Synthetic, seeded cloud networks
- Weighted server network with link latencies
- Multiple tasks with demand, value and origin
- Server capacity constraints
- Latency penalty in the utility
- Minimum spanning tree backbone
- Shortest-path latency matrix
- Greedy task proposals
- Knapsack selection on each server
- Comparison with baselines and the exact optimum
- Performance experiments and charts

The current system does not include real network traces, several resource types per task, tasks arriving over time, task migration, link failures, or replication of tasks on several servers.

1.4 Key Terms

Table 1.1: Key Terms

Term	Meaning
Server	A machine or data centre that can run tasks. A node of the graph.
Link	A network connection between two servers, with a latency in milliseconds.
Latency	Delay in milliseconds. Between two servers it is the shortest-path length.
Task	A unit of work (for example a virtual machine) with a demand, a value and an origin.
Demand	How many capacity units a task needs on its server.
Origin	The server where the task's users connect to the cloud.
Utility	Value of a task minus the latency penalty for the chosen server.
Load	Total task demand divided by total capacity. Above 1, not every task can be placed.
MST	Minimum spanning tree: the cheapest set of links that connects all servers with no loops.
Heuristic	A fast method that gives a good answer but is not guaranteed to be the best.
NP-hard	A problem for which no fast exact algorithm is known for all inputs.

CHAPTER 2 — PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

The cloud environment is represented as a weighted graph:

$$\mathbf{G = (V, E)}$$

where V represents servers and E represents network links, each with a latency in milliseconds. The latency between two servers a and b, written L(a, b), is the length of the shortest path between them. Each server s has a capacity cap(s).

Each task is represented as:

$$\mathbf{T_i = (d_i, p_i, o_i)}$$

where:

- d_i = demand (capacity units needed)
- p_i = value of the task
- o_i = origin server, where its users connect

Placing task i on server s earns the utility:

$$\mathbf{u(i, s) = p_i - \lambda \times L(o_i, s)}$$

where λ is the value lost per millisecond of latency (λ = 1 unless stated otherwise). The objective is to decide which tasks to run and where, so that total utility is as large as possible while no server is overloaded.

The main objective is:

$$\mathbf{Maximize\ \Sigma\ u(i, s) \times x(i, s)}$$

subject to:

$$\mathbf{\Sigma_s\ x(i, s) \leq 1 \quad for\ every\ task\ i}$$

$$\mathbf{\Sigma_i\ d_i \times x(i, s) \leq cap(s) \quad for\ every\ server\ s}$$

where x(i, s) = 1 if task i runs on server s and 0 otherwise. A task is placed only if its utility is positive.

Hardness. With a single server the problem is exactly the 0-1 knapsack problem, which is NP-hard [10]. The full problem is a generalised assignment problem and also NP-hard, so no fast

exact algorithm is known. This is why the project uses a heuristic and measures its quality against the exact optimum on small instances instead of claiming optimality.

2.2 Objectives

Table 2.1: Project Objectives

Objective	Description
O1	Build a low-cost backbone network (MST)
O2	Find shortest latencies between servers
O3	Let each task choose its best server
O4	Select the most valuable tasks on each server
O5	Respect server capacities
O6	Compare with baselines and the exact optimum
O7	Analyse correctness and complexity
O8	Measure runtime and solution quality
O9	Study sensitivity to load, latency penalty and backbone

2.3 Constraints

The main constraints are:

Capacity

$\Sigma_i d_i \times x(i, s) \leq cap(s)$

Single placement

$\Sigma_s x(i, s) \leq 1$

Positive utility

$u(i, s) > 0$ for every placed task

Integer demands and capacities

d_i and $cap(s)$ are whole numbers, so that the knapsack table can be indexed by capacity

2.4 Assumptions

- All data is synthetic and generated from fixed random seeds, so every result can be reproduced exactly.
- The workload is static: all tasks are known before placement starts and tasks are never moved.
- Latency is the shortest-path length and link latencies do not change.

- A task runs on at most one server.

CHAPTER 3 — LITERATURE REVIEW

3.1 Algorithmic Foundations

The project builds on classical algorithms that are usually taught separately [1], [2]. Table 3.1 lists the sources and the part of the project each one supports.

Table 3.1: Sources and Their Role in the Project

Source	Contribution	Used for
Kruskal (1956) [3]	Greedy minimum spanning tree using sorted edges and union-find	Backbone network
Prim (1957) [4]	Minimum spanning tree grown from one node	Cross-check of Kruskal
Dijkstra (1959) [5]	Shortest paths for non-negative weights	Server-to-server latency
Bellman (1958) [6]	Shortest paths with negative edges	Penalty and credit edges, cycle check
Floyd (1962) [7], Warshall (1962) [8]	All-pairs shortest paths by dynamic programming, $O(V^3)$	Latency matrix, dense networks
Johnson (1977) [9]	All-pairs shortest paths for sparse graphs by reweighting	Latency matrix, sparse networks
Martello and Toth (1990) [10]	Knapsack problems: dynamic programming and bounds	Task selection on each server
Cormen et al. [1], Kleinberg and Tardos [2]	Textbook proofs, complexity and greedy exchange arguments	Correctness arguments

3.2 Minimum Spanning Trees

A spanning tree connects all servers with exactly $V - 1$ links and no loops. The minimum spanning tree (MST) is the spanning tree with the smallest total latency, so it is the cheapest network that still reaches every server. Kruskal [3] sorts links by weight and adds a link when it joins two separate groups. Prim [4] grows a tree from one server by always adding the cheapest link that leaves the tree. Both are correct because of the cut property: the cheapest link crossing any split of the servers belongs to some minimum spanning tree.

3.3 Shortest Path Algorithms

Dijkstra's algorithm finds shortest paths in graphs with non-negative edge weights [5]. In this project it is used to calculate latency between servers. Its complexity using an adjacency list and a priority queue is:

O((V + E) log V)

Bellman-Ford [6] allows negative weights in O(VE) time. Floyd-Warshall [7], [8] computes all pairs in $O(V^3)$ time, which suits dense networks. Johnson's algorithm [9] reweights the graph and runs Dijkstra from every server in $O(VE + V^2 \log V)$ time, which suits sparse networks.

3.4 Knapsack Problem

The 0/1 Knapsack problem selects items subject to a capacity constraint while maximizing value [10].

The project maps:

Table 3.2: Knapsack and Cloud Placement Mapping

Knapsack	Cloud Task Placement
Item	Task
Weight	Task demand
Value	Task utility on that server
Capacity	Remaining server capacity

Knapsack solves the selection problem for one server but does not decide which server each task should ask for. Therefore, it is combined with graph algorithms and greedy proposals.

3.5 Related Optimisation Problems

Bin packing and the generalised assignment problem. Bin packing asks to fit items into the fewest bins. The generalised assignment problem (GAP) asks to assign each item to at most one of several bins, where the profit of an item depends on the bin, so that no bin overflows. Task placement is a GAP: the bins are servers, and the profit of putting a task on a server is its utility, which depends on latency. GAP is NP-hard [10].

Facility location. Choosing where to put servers or replicas so that users are close to them is studied as facility location and k-median problems. Our setting is different because the servers already exist and the question is which tasks to run on them, but the same idea of distance-based cost appears.

3.6 Research Gap

Existing research on placement and assignment includes mathematical optimization, approximation algorithms and heuristics [1], [2], [10].

This project focuses on an educational implementation that combines **graph algorithms, greedy methods, dynamic programming and exact optimization** in one explainable system, checks each algorithm against another or against an exact solution, and measures which choices matter. In particular it measures the value of the dynamic programming step against a greedy rule, and the effect of using a spanning tree as the only network on placement quality.

CHAPTER 4 — METHODOLOGY AND SYSTEM DESIGN

4.1 Proposed Architecture

The system follows this pipeline:

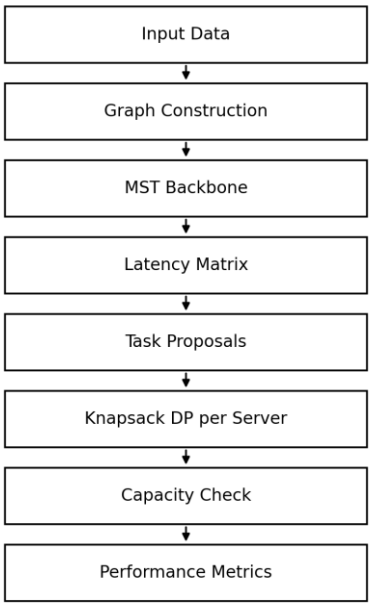


Figure 4.1: Proposed System Architecture

4.2 Input Data

The project uses three main entities:

Server

- Server ID
- Capacity
- Position (for synthetic networks)

Task

- Task ID
- Demand
- Value
- Origin server

Graph

- Source server
- Destination server
- Latency (ms)

4.3 Sample Network

A six-server network is used throughout Chapter 6 to show every step by hand. Table 4.1 lists the server capacities, Table 4.2 the links and Table 4.3 the tasks.

Table 4.1: Sample Servers

Server	Capacity
0	5
1	3
2	4
3	4
4	3
5	3

Table 4.2: Sample Graph

Link	Latency (ms)
0 - 1	4
0 - 2	2
1 - 2	1
1 - 3	5
2 - 3	8
2 - 4	10
3 - 4	2
3 - 5	6
4 - 5	3

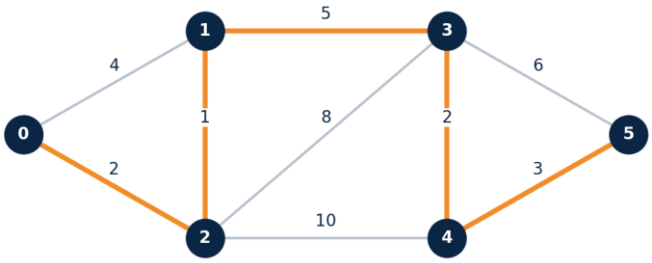


Figure 4.2: Sample Six-Server Network (orange links form the MST)

Table 4.3: Sample Tasks

Task	Demand	Value	Origin
0	3	30	0
1	4	25	0
2	2	20	1
3	3	35	3
4	4	28	4
5	2	15	5

4.4 Dataset Generation for Experiments

Network. Servers receive random positions on a plane. The latency of a link is 1 ms plus 40 ms times the distance plus 0 to 2 ms of jitter. Every server is linked to its nearest earlier server, which guarantees a connected network. A density factor adds links from each server to its nearest neighbours (three per unit of the factor).

Workload. Task demand is an integer from 1 to 10, task value from 5 to 60, and each task's users sit at a random server. Server capacities vary between 0.6 and 1.4 times the average. The load is set by scaling the total capacity.

Table 4.4: Dataset Parameters

Parameter	Value or range
Servers	10 to 200, depending on the experiment
Tasks	16 to 600
Link latency	1 ms + 40 ms × distance + 0 to 2 ms jitter
Links per server	Nearest earlier server plus three nearest neighbours per unit of density factor
Task demand	Integer 1 to 10
Task value	Integer 5 to 60
Server capacity	0.6× to 1.4× of the average, at least 5
Load	0.4 to 2.0
λ	1 (0.25 to 8 in the sensitivity study)

4.5 Minimum Spanning Tree Algorithms

Kruskal's algorithm

```
sort all links by latency
put every server in its own group
for each link (u, v, w) in sorted order:
```

```
    if group(u) != group(v):          # the link joins two separate groups
        add the link to the tree
        merge group(u) and group(v)
```

Sorting costs O(E log E). The group checks use union-find with union by rank and path halving, which is nearly constant time per operation. The total is O(E log E).

Prim's algorithm

```
start with one server in the tree; put its links on a min-heap
while the tree has fewer than V servers:
    pop the cheapest link (u, v, w)
    if v is already in the tree: continue
    add v and the link to the tree
    push the links of v to servers not yet in the tree
```

With a binary heap [11] the time is O(E log V). Both algorithms give a tree of the same total weight, which the tests verify.

Why they are correct. Both rely on the cut property. Suppose a minimum tree T does not contain the cheapest link e crossing a split of the servers. Adding e to T creates a loop, and the loop must cross the split a second time through some link f that is at least as costly. Swapping f for e gives a spanning tree that is no heavier, so a minimum tree containing e exists. Kruskal adds a link only when it joins two groups, and Prim adds the cheapest link leaving the tree, so each step is safe and after V − 1 steps the tree is minimum.

4.6 Shortest Path Algorithms

Dijkstra

```
dist[source] = 0, all other dist = infinity
push (0, source) on a min-heap
while the heap is not empty:
    (d, u) = pop the smallest entry
    if d > dist[u]: continue           # outdated entry
    for each link (u, v, w):
        if d + w < dist[v]:
            dist[v] = d + w
            push (dist[v], v)
```

Time is O((V + E) log V). When a server u is popped with the smallest tentative distance, dist[u] is already the true shortest distance: a shorter path would leave the finished set through some unfinished server whose distance is already smaller, which contradicts the choice of u. This argument needs non-negative weights, which holds for latencies.

Bellman-Ford relaxes every link V − 1 times in O(VE) time. After k rounds the distances are correct for all paths with at most k links, and a shortest path has at most V − 1 links. If a link can still be relaxed afterwards, there is a negative cycle.

Floyd-Warshall computes all pairs with the recurrence:

$$\mathbf{d(k+1, i, j) = \min(d(k, i, j),\ d(k, i, k) + d(k, k, j))}$$

where d(k, i, j) is the shortest distance using only servers numbered below k as intermediate stops. Time is $O(V^3)$ whatever the number of links.

Johnson's algorithm adds a virtual server joined to all others by zero-weight links, runs Bellman-Ford from it to get a number h(v) for every server, and gives each link the new weight $w(u, v) + h(u) - h(v)$, which is never negative. Every path from s to t changes by the same amount $h(s) - h(t)$, so shortest paths stay the same. It then runs Dijkstra from every server in $O(VE + V^2 \log V)$ time.

Choice in the code. The program uses Johnson when the number of links is at most $V^2 / 8$ and Floyd-Warshall otherwise. Chapter 6 measures the difference.

4.7 Knapsack Algorithm

The 0/1 Knapsack DP uses:

O(nC)

time and:

O(nC)

space, for n items and capacity C. Let best(i, c) be the greatest value obtainable from the first i items with capacity c. Then:

best(i, c) = max(best(i−1, c), best(i−1, c − w(i)) + v(i))

The chosen items are recovered by walking back from the last cell. It is exact because an optimal choice for the first i items contains an optimal choice for the first $i - 1$ items with the remaining capacity.

Why a greedy rule is not enough. Sorting items by value per unit of weight and taking them while they fit can fail. Take capacity 10 and three items: A with weight 6 and value 60 (density 10), B with weight 5 and value 45 (density 9), and C with weight 5 and value 45 (density 9). The greedy rule takes A and then nothing else fits, for a total of 60. The dynamic program takes B and C for a total of 90. Section 6.12 measures how much this matters in placement.

The fractional knapsack (taking items by density and allowing fractions) is optimal for the divisible case and gives an upper bound for the 0-1 case. Job sequencing with deadlines (profit-ordered greedy into the latest free slot) is also implemented for batch jobs with deadlines.

4.8 Greedy Proposal Algorithm

Each pending task computes the utility u(i, s) of every server it has not been rejected by, and proposes to the server with the highest positive utility. Letting every task go to its best server first is fast and keeps tasks close to their users.

The greedy step alone does not guarantee the best overall combination, because several tasks may propose to the same server. That is why each server then solves a knapsack over its proposals.

4.9 Placement Algorithm

The placement procedure works in rounds, as shown in Algorithm 1 and Figure 4.3.

```
while some tasks are pending:
    for each pending task t:
        s = best server for t: highest u(t, s) > 0 and not banned for t
        if no such server: drop t           # it cannot earn positive utility
        else: add t to the proposals of s
    for each server s with proposals:
        run 0-1 knapsack DP over its proposals
            (value = u(t, s), weight = d(t), capacity = remaining capacity of s)
        place the chosen tasks on s and reduce its remaining capacity
        ban every rejected task from s
```

Algorithm 1: Greedy proposals with knapsack selection

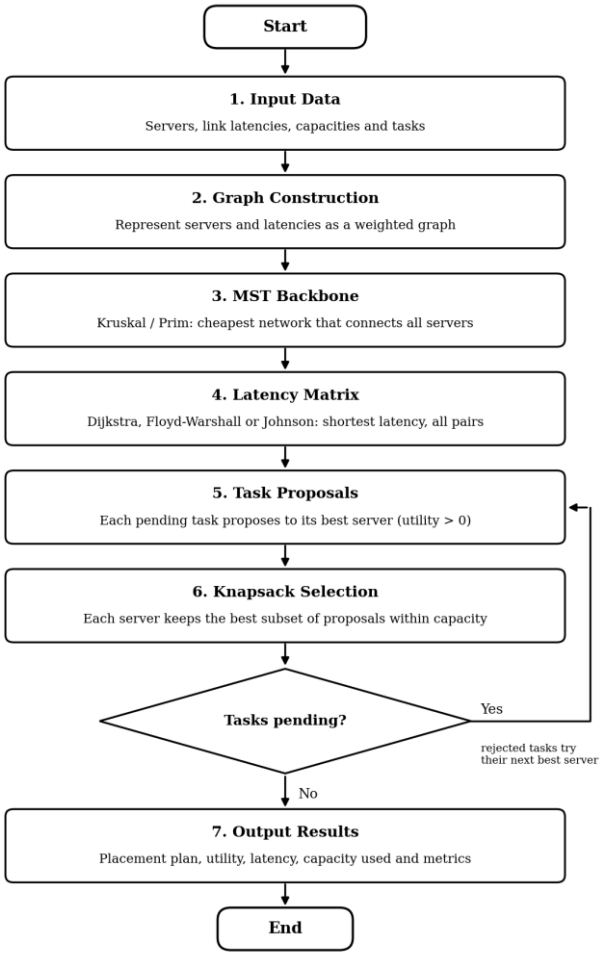


Figure 4.3: Placement Algorithm Workflow

Feasibility. A server accepts only a subset that the knapsack guarantees fits within its remaining capacity, so no server is ever overloaded. The code also checks this for every plan it produces.

Termination. In every round each pending task is either placed, dropped, or banned from the server it proposed to. A task can be banned from at most |S| servers, so after |S| rounds it has no server left and is dropped in the next round. The loop therefore ends after at most |S| + 1 rounds.

Quality. For the proposals a server receives in a round, its choice is optimal because the dynamic program is exact. The overall plan is not guaranteed to be optimal, because a task may be accepted by a server where it blocks a larger task that would have been better placed there. Section 6.5 shows this happening.

Complexity. Finding proposals costs O(|T| |S|) per round. The knapsack runs cost O(n(s) C(s)) for server s, where n(s) is its number of proposals and C(s) its remaining capacity, and the

proposals sum to at most |T| per round. With at most |S| + 1 rounds the total is O((|S| + 1)(|T| |S| + |T| C_max)), where C_max is the largest capacity.

4.10 Baselines and Exact Solver

Nearest-first sorts tasks by value and places each on the lowest-latency server that has room. **Random** places tasks in random order on random servers with room. **Greedy density selection** is the placement algorithm with the knapsack on each server replaced by taking proposals in order of utility per unit of demand; it measures the value of the dynamic program. The **exact solver** is the integer program of Section 2.1, solved with SciPy [12]; it is only practical for small instances.

4.11 Validation

After every run the program checks:

- Capacity of every server is not exceeded
- Each task is assigned to at most one server
- Utility, percentage placed, average latency and capacity used are computed from the plan itself

4.12 Complexity Summary

Table 4.5: Algorithms and Their Time Complexity

Subproblem	Algorithm	Time
Backbone	Kruskal / Prim	O(E log E) / O(E log V)
Latency from one server	Dijkstra	O((V + E) log V)
Negative links	Bellman-Ford	O(VE)
All pairs, dense	Floyd-Warshall	$O(V^3)$
All pairs, sparse	Johnson	$O(VE + V^2 \log V)$
Selection bound	Fractional knapsack	O(n log n)
Selection on a server	0-1 knapsack DP	O(nC)
Deadline jobs	Job sequencing	O(nD), D = largest deadline
Placement	Greedy proposals + DP	O((|S| + 1)(|T||S| + |T| C_max))

CHAPTER 5 — IMPLEMENTATION

5.1 Technologies Used

Table 5.1: Technologies Used

Component	Technology
Language	Python
Graph	Custom adjacency-list graph
Priority queue	Python heapq [11]
Backbone	Kruskal and Prim
Shortest path	Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson
Selection	Greedy proposals
Optimization	0/1 Knapsack DP
Exact solver	SciPy integer programming [12]
Data	Seeded synthetic generator, JSON results
Visualization	Matplotlib [13]
Testing	Python test script (pytest compatible)

5.2 Project Modules

The implementation is divided into:

- **graph.py:** adjacency-list graph, topology and workload generators
- **mst.py:** union-find, Kruskal, Prim
- **shortest.py:** Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson
- **greedy.py:** fractional knapsack bound, job sequencing
- **dp.py:** 0-1 knapsack with recovery of the chosen items
- **placement.py:** greedy proposals with DP, baselines, exact solver, metrics
- **run_experiments.py** and **extra_experiments.py:** all experiments, charts and result files
- **tests/test_all.py:** correctness tests

This modular structure makes the system easier to test and maintain. The six core source files total about 420 lines of Python.

5.3 Implementation Notes

Table 5.2: Implementation Notes

Topic	What the code does
Union-find	Union by rank and path halving, so each find is nearly constant time.
Dijkstra	Skips outdated heap entries instead of updating them in place.
All-pairs choice	Picks Johnson when links are at most $V^2 / 8$, otherwise Floyd-Warshall.
Johnson	Treats each link as two directed links and adds a virtual source for Bellman-Ford.
Knapsack	Stores the full table so the chosen items can be recovered.
Placement	Keeps a banned set per task and remaining capacity per server; checks capacity on every plan.
Selector option	solve_greedy_dp takes a selector argument ('dp' or 'density') for the ablation study.

5.4 Experiment Runner and Outputs

Running run_experiments.py and extra_experiments.py regenerates every chart, table and result file used in this report. Each experiment uses fixed random seeds. Figure 5.1 shows the output of one placement run on a 40-server network: the MST backbone is drawn in orange and the darker a server, the fuller it is.

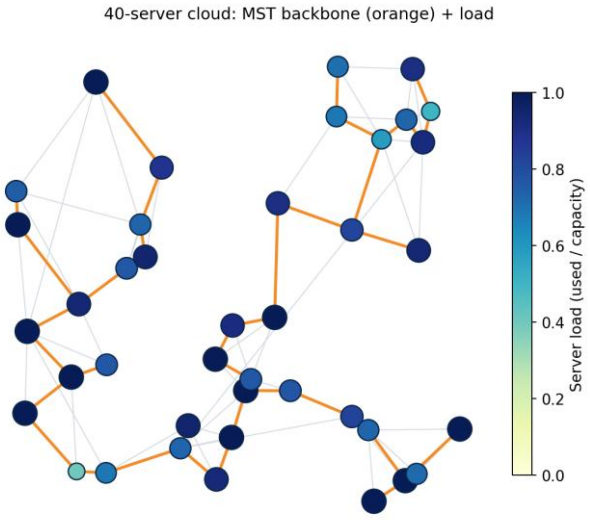


Figure 5.1: Placement Output on a 40-Server Network

5.5 Testing

The project includes tests for the backbone, shortest paths, knapsack, scheduling and placement.

Table 5.3: Automated Tests

Test	What it checks
Kruskal equals Prim	Equal total weight and V − 1 links on five random networks
Shortest paths agree	Dijkstra, Bellman-Ford, Floyd-Warshall and Johnson give the same distances on a 30-server network
Johnson, negative edges	Correct distances on a graph with a negative link
Negative cycle	Bellman-Ford reports a cycle of total weight below zero
Knapsack vs brute force	DP equals exhaustive search on 30 random cases; fractional bound is never lower
Job sequencing	Returns the known optimal profit (142) on a textbook example
Placement quality	Plan is feasible, does not exceed the exact optimum, and reaches at least 85% of it

The project achieved:

7 tests passed

This confirms successful execution of the defined test cases.

CHAPTER 6 — RESULTS AND DISCUSSION

6.1 Worked Example on the Sample Network

The six-server sample network of Section 4.3 is followed step by step. First the minimum spanning tree is built with Kruskal's algorithm. The links are sorted by latency and processed in order, and a link is added only if its two ends are in different groups.

Table 6.1: Kruskal's Algorithm on the Sample Network

Step	Link	Latency	Action	Reason
1	1 - 2	1	Add	Joins two groups
2	0 - 2	2	Add	Joins two groups
3	3 - 4	2	Add	Joins two groups
4	4 - 5	3	Add	Joins two groups
5	0 - 1	4	Skip	Ends already connected (loop)
6	1 - 3	5	Add	Joins two groups
7	3 - 5	6	Skip	Ends already connected (loop)
8	2 - 3	8	Skip	Ends already connected (loop)
9	2 - 4	10	Skip	Ends already connected (loop)

The tree has five links with total latency 13, against 41 for all nine links, a saving of 68.3% of link cost.

Next, shortest paths from server 0 are found with Dijkstra's algorithm.

Table 6.2: Dijkstra's Algorithm from Server 0

Server	Distance (ms)	Shortest path
0	0	0
1	3	0 - 2 - 1
2	2	0 - 2
3	8	0 - 2 - 1 - 3
4	10	0 - 2 - 1 - 3 - 4
5	13	0 - 2 - 1 - 3 - 4 - 5

Server 1 is reached more cheaply through server 2 (2 + 1 = 3) than directly (4). Repeating from every server, or running Floyd-Warshall, gives the full latency matrix.

Table 6.3: Latency Matrix L(a, b) in Milliseconds

	0	1	2	3	4	5
0	0	3	2	8	10	13
1	3	0	1	5	7	10
2	2	1	0	6	8	11
3	8	5	6	0	2	5
4	10	7	8	2	0	3
5	13	10	11	5	3	0

6.2 Optimized Result

The Hybrid optimizer placed:

Table 6.4: Optimized Placement Result

Task	Server	Demand	Utility	Latency (ms)
0	0	3	30	0
1	2	4	23	2
2	1	2	20	0
3	3	3	35	0
5	5	2	15	0
Total		14	123	

Task 4 (demand 4, value 28) could not be placed: every server that could hold 4 units was already used by smaller tasks.

6.3 Generated Placement Rounds

Utility is value minus latency, so task 0 earns 30 on server 0 and 28 on server 2. Each row below is one server's decision in one round.

Table 6.5: Proposal Rounds on the Sample Network

Round	Server	Space left	Proposals (utility, demand)	Accepted	Rejected
1	0	5	task 0 (30, 3), task 1 (25, 4)	task 0	task 1
1	1	3	task 2 (20, 2)	task 2	none
1	3	4	task 3 (35, 3)	task 3	none
1	4	3	task 4 (28, 4)	none	task 4
1	5	3	task 5 (15, 2)	task 5	none
2	2	4	task 1 (23, 4)	task 1	none
2	3	1	task 4 (26, 4)	none	task 4
3	5	1	task 4 (25, 4)	none	task 4
4	1	1	task 4 (21, 4)	none	task 4
5	2	0	task 4 (20, 4)	none	task 4
6	0	2	task 4 (18, 4)	none	task 4

In round 1, tasks 0 and 1 both propose to server 0, which has room for only one of them. The knapsack keeps task 0 (utility 30) and rejects task 1, which moves to server 2 in round 2. Task 4 needs 4 units, but its best server (server 4) has only 3, so it is rejected, and it then tries servers 3, 5, 1, 2 and 0 in turn. None has 4 units free any more, so after round 6 it has no server left and is dropped in round 7. The run uses 7 rounds, exactly the bound $|S| + 1$.

For server 0 in round 1 the knapsack table (capacity 5, task 0 with demand 3 and utility 30, task 1 with demand 4 and utility 25) is:

Table 6.6: Knapsack Table for Server 0, Round 1

Items considered	c = 0	c = 1	c = 2	c = 3	c = 4	c = 5
none	0	0	0	0	0	0
task 0	0	0	0	30	30	30
tasks 0, 1	0	0	0	30	30	30

The best value at capacity 5 is 30: the two tasks together need 7 units, so only task 0 fits.

6.4 Performance

Table 6.7: Performance on the Sample Network

Metric	Result
Total Utility	123
Placed Tasks	5 of 6 (83.3%)
Average Latency	0.4 ms
Capacity Used	63.6%
Rounds Used	7
Capacity Violations	0

Total demand placed is 14 units out of a total capacity of 22: (14 / 22) × 100 = 63.6%.

6.5 Algorithm Comparison

Table 6.8: Algorithm Comparison on the Sample Network

Method	Placed tasks	Utility	% of optimum	Feasible
Nearest-first	0, 2, 3, 4, 5	120	81.6	Yes
Greedy + DP	0, 1, 2, 3, 5	123	83.7	Yes
Exact optimum	0, 1, 2, 3, 4, 5	147	100.0	Yes

The exact optimum places all six tasks. It does so by moving task 0 from server 0 to server 2 and task 3 from server 3 to server 4, each at a cost of 2 ms, which frees servers 0 and 3 for the larger tasks 1 and 4. The proposal algorithm cannot see this, because each task goes to its own best server first, and the small tasks fill the space the large ones needed. It loses task 4 (value 28) and ends at 123 against 147. This small case shows both the strength and the limit of the method: it is fast and keeps tasks near their users, but it is a heuristic.

6.6 Algorithm Runtime Experiment

Table 6.9: Runtime Against Network Size (ms)

Servers	25	50	100	150	200
Kruskal	0.03	0.04	0.09	0.13	0.18
Prim	0.02	0.04	0.08	0.13	0.3
Dijkstra, all sources	0.29	1.17	5.41	14.09	23.91
Floyd-Warshall	0.56	4.08	33.27	132.59	303.2
Johnson	0.36	1.38	5.81	12.7	24.54

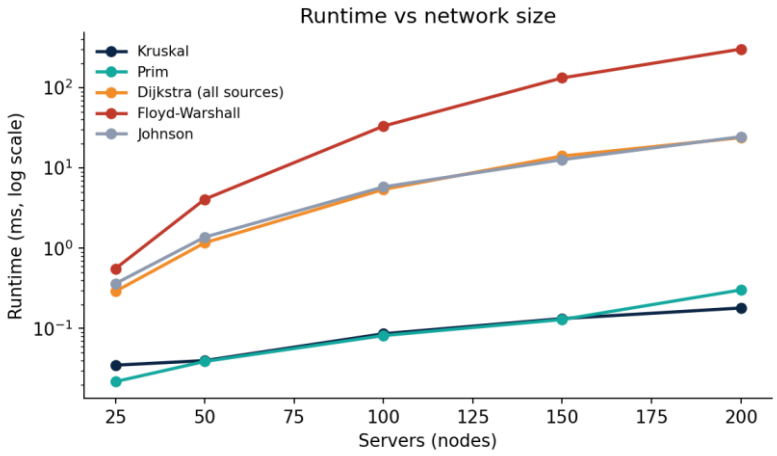


Figure 6.1: Runtime Against Network Size (Log Scale)

Kruskal and Prim stay below a millisecond even at 200 servers. At 200 servers Johnson took about 25 ms against about 303 ms for Floyd-Warshall, roughly 12 times faster. Johnson's growth matches Dijkstra run from every server, as the theory predicts. Floyd-Warshall grows by roughly a factor of eight each time the size doubles, which is the cubic growth of $O(V^3)$. All timings are single measurements on one machine.

6.7 Sparse and Dense Networks

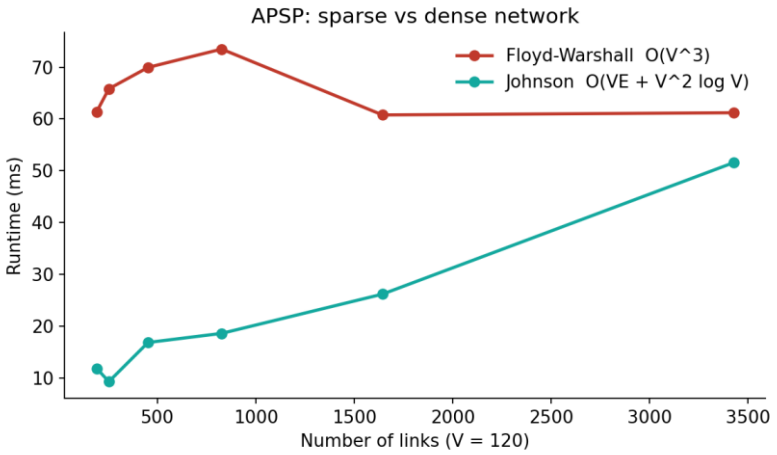


Figure 6.2: All-Pairs Runtime Against Number of Links (120 Servers)

Table 6.10: All-Pairs Runtime on 120 Servers (ms)

Links	191	253	452	826	1643	3427
Floyd-Warshall	61.4	65.8	70	73.5	60.8	61.2
Johnson	11.9	9.4	16.9	18.7	26.2	51.6

Floyd-Warshall's time stays roughly constant at about 60 to 74 ms because it does not depend on the number of links. Johnson grows from about 12 ms at 191 links to about 52 ms at 3427

links. Johnson was faster in every case tested, but its lead shrank from about 5 times to 1.2 times. The theory says Floyd-Warshall wins only when the network is close to complete (about 7000 links for 120 servers), which the tests did not reach.

6.8 Solution Quality

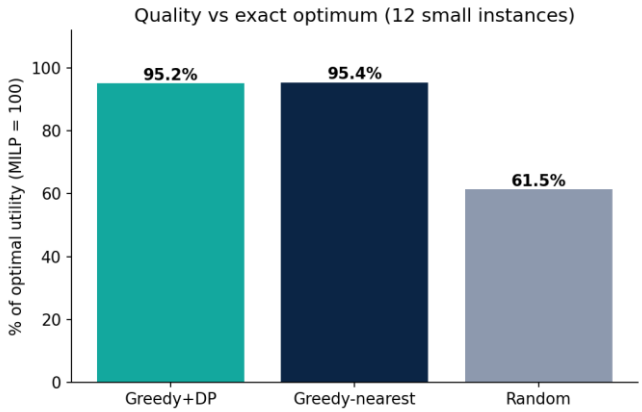


Figure 6.3: Utility as a Percentage of the Exact Optimum (12 Small Instances)

Table 6.11: Quality Against the Exact Optimum

Method	Mean % of optimum	Worst case %
Greedy + DP	95.2	90.9
Nearest-first	95.4	88.0
Random	61.5	47.4

The setup is 10 servers, 16 tasks and 12 instances, with the exact optimum from the integer program. Greedy + DP reaches about 95% of the optimum, almost the same as nearest-first (95.4%), and far above random placement (61.5%). On small, lightly loaded instances the dynamic program therefore gives no clear gain in utility. Its benefit appears when the cloud is overloaded.

6.9 Load Experiment

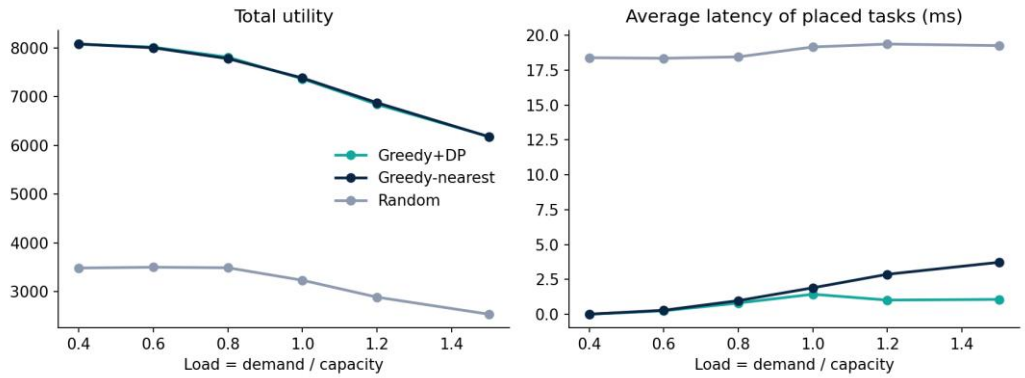


Figure 6.4: Total Utility and Average Latency as Load Increases

Table 6.12: Results on 60 Servers and 250 Tasks (Average of 4 Seeds)

Load	0.4	0.6	0.8	1.0	1.2	1.5
Utility, Greedy + DP	8080.4	8016.0	7810.3	7367.3	6842.4	6182.1
Utility, nearest-first	8080.4	8003.7	7781.2	7384.2	6876.8	6179.1
Latency ms, Greedy + DP	0.01	0.25	0.81	1.44	1.02	1.07
Latency ms, nearest-first	0.01	0.29	0.98	1.90	2.87	3.73
Placed %, Greedy + DP	100.0	99.6	96.8	90.7	80.6	70.6
Placed %, nearest-first	100.0	99.4	96.5	88.7	79.2	67.9

Utility is almost identical for the two methods, and nearest-first is slightly higher at loads 1.0 and 1.2. At load 1.5, Greedy + DP keeps the average latency at 1.07 ms against 3.73 ms for nearest-first (about 3.5 times lower) and places 70.6% of the tasks against 67.9%. Utility falls for both methods as load grows, from about 8080 at load 0.4 to about 6180 at load 1.5, because tasks start to be turned away.

A likely reason is how the two methods handle a full server. Nearest-first sends each task to the closest server that still has room, so once the near servers fill up, later tasks are sent further away. Greedy + DP lets each server pick the best subset of the tasks that want it, and sends the rest to their next best choice, so the tasks that stay close are the ones for which closeness is worth the most.

6.10 Backbone Experiment

Table 6.13: MST Backbone Against the Full Network

Servers	Links (full)	Links (MST)	Link saving	Cost (full)	Cost (MST)	Cost cut	Latency stretch
30	109	29	73.4%	1280.8	210.3	83.6%	1.76x
60	228	59	74.1%	1919.6	334.4	82.6%	2.42x
100	371	99	73.3%	2590.7	469.4	81.9%	2.68x

The MST keeps only $V - 1$ links, about 74% fewer than the full network, and cuts the total link cost by 82 to 84%. The price is longer routes: the average pairwise latency is 1.8 to 2.7 times higher when traffic may use only the tree. This figure averages over all pairs of servers, including far-apart pairs. The next section shows how much it matters for placement.

6.11 Placement over the MST Only

To connect the backbone to placement directly, the placement algorithm was run twice on the same networks and workloads: once with latencies from the full network, and once with latencies computed using only the tree links (load 0.9, average of 4 seeds).

Table 6.14: Placement with Full-Network Latencies Against Tree-Only Latencies

Servers	Tasks	Utility (full)	Utility (MST only)	Utility lost	Latency full (ms)	Latency MST (ms)
30	120	3567.8	3542.5	0.7%	1.08	1.10
60	250	7735.9	7653.4	1.1%	1.09	1.05
100	400	12776.1	12618.2	1.2%	1.07	1.05

Utility is only 0.7% to 1.2% lower when traffic is restricted to the tree, and the average latency of placed tasks is about the same (1.05 to 1.10 ms). A likely reason is that placement mostly puts tasks on or next to their origin server, where the tree route and the shortest route are almost the same, so the tree's longer routes mainly affect far-apart pairs that few placements use. This explanation was not tested directly. In this model, the MST is therefore a practical backbone, and its 82-84% saving in link cost costs very little in placement quality.

6.12 Value of the Dynamic Program

To measure what the knapsack dynamic program adds, it was replaced on every server by a greedy rule that takes proposals in order of utility per unit of demand, and everything else was left unchanged (60 servers, 250 tasks, average of 6 seeds).

Table 6.15: Dynamic Programming Against a Greedy Selection Rule

Load	Utility (DP)	Utility (greedy)	DP advantage	Placed % (DP)	Placed % (greedy)
0.8	7788.8	7773.8	0.2%	97.3	97.4
1.0	7352.0	7288.8	0.9%	90.9	90.4
1.2	6859.5	6762.2	1.4%	81.5	80.9
1.5	6179.8	6062.3	1.9%	71.5	71.3
2.0	5185.3	5028.1	3.1%	58.7	58.7

The dynamic program gives higher utility at every load, by 0.2% at load 0.8 and by 3.1% at load 2.0. The advantage grows with load because servers receive more proposals than they can hold, so the choice of subset matters more. The gain is modest but consistent. A plausible reason is that many small tasks are available, which is the case where a greedy rule comes close to the optimum, but this was not tested.

6.13 Latency Penalty Experiment

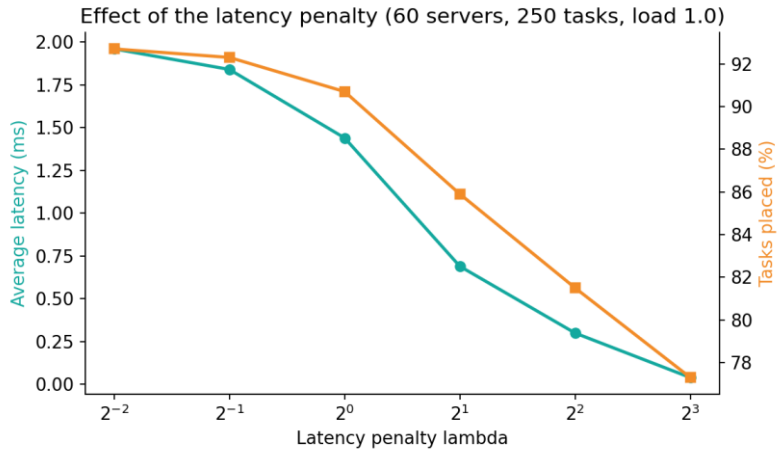


Figure 6.5: Effect of the Latency Penalty on Average Latency and Tasks Placed

Table 6.16: Results for Different Values of λ (60 Servers, 250 Tasks, Load 1.0)

λ	0.25	0.5	1	2	4	8
Average latency (ms)	1.96	1.84	1.44	0.69	0.30	0.04
Tasks placed (%)	92.7	92.3	90.7	85.9	81.5	77.3
Task value kept (%)	95.6	95.7	95.2	92.4	89.4	85.5

A larger λ makes latency more expensive. Average latency falls from 1.96 ms at λ = 0.25 to 0.04 ms at λ = 8, but fewer tasks are placed (92.7% down to 77.3%) and the share of total task value that is kept falls from 95.6% to 85.5%. The operator therefore chooses λ according to how much response time is worth compared to running more tasks. The default λ = 1 is a middle setting, not a recommended value for any real cloud.

6.14 Rounds and Scalability

Table 6.17: Rounds Used by the Placement Algorithm (Average of 4 Seeds)

Servers	Tasks	Load	Average rounds	Most rounds	Bound |S| + 1
20	100	1.0	11.2	20	21
40	200	1.2	36	41	41
60	400	1.5	56.8	61	61
80	600	1.5	76.8	80	81

The number of rounds never exceeded the bound |S| + 1. Under overload it comes close to the bound, mainly because tasks that fit nowhere keep proposing until every server has rejected them, as in the worked example. This does not make the algorithm slow, because the later rounds involve very few tasks.

Table 6.18: Placement Time at Load 0.9

Servers	Tasks	Time (s)
20	100	0.0026
40	200	0.0047
60	400	0.0087
80	600	0.0186

Placing 600 tasks on 80 servers takes under 0.02 seconds, so the heuristic is fast enough for much larger instances than the exact solver can handle.

6.15 Summary of Findings

Table 6.19: Summary of Experimental Findings

Question	Finding
Which all-pairs algorithm?	Johnson on sparse networks: about 12 times faster than Floyd-Warshall at 200 servers.
How good is Greedy + DP?	About 95% of the exact optimum on small cases, the same as nearest-first there.
When does it help?	Under overload: average latency about 3.5 times lower than nearest-first at load 1.5.
What does the DP add?	0.2% to 3.1% more utility than a greedy selection rule, growing with load.
Does the MST hurt placement?	Link cost falls 82-84%; utility with tree-only routes is 0.7-1.2% lower.
How sensitive is it to λ?	Strongly: latency falls and fewer tasks are placed as λ grows.
Is the heuristic optimal?	No. On the sample network it reached 83.7% of the optimum.

Threats to validity. The data is synthetic, so real clouds with regions, racks and bursty demand may behave differently. The exact comparison is limited to about 10 servers and 16 tasks. Timings are single runs, and load and backbone results average only 4 to 6 seeds. The model has one resource type, a static workload and latency as the only network cost, so bandwidth limits, queueing and link failures are not modelled.

CHAPTER 7 — CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project successfully developed a hybrid system for cloud task placement under capacity and latency constraints.

The minimum spanning tree gives a cheap backbone, shortest-path algorithms compute latency, greedy proposals let tasks ask for their best server, and Knapsack DP selects the best subset of tasks on each server. The measurements showed where each choice wins.

The main results were:

- **12× faster** all-pairs latency with Johnson than Floyd-Warshall at 200 servers
- **About 95%** of the exact optimum on small instances
- **About 3.5× lower** average latency than nearest-first at load 1.5
- **82-84%** lower link cost with an MST backbone, at only 0.7-1.2% lower utility
- **0.2-3.1%** more utility from the dynamic program than from a greedy selection rule

The project demonstrates that combining multiple classical algorithms can provide a practical and explainable solution to a constrained optimization problem, and that measuring them shows where each one is worth its cost.

7.2 Limitations

The current system:

- Uses synthetic data.
- Uses one resource demand per task.
- Uses a static workload with no arrivals or migration.
- Does not model link failures.
- Does not replicate tasks on several servers.
- Does not guarantee global optimality.
- Compares with the exact optimum only on small instances.

7.3 Future Scope

Future versions can include:

1. Several resources per task (CPU and memory) with a multi-dimensional knapsack.
2. Real network topologies and workload traces.
3. Tasks that arrive and leave over time, with migration.
4. Link failures, and finding the cheapest extra links that make the MST backbone resilient.
5. Replica placement, so that a task can run on several servers.
6. Approximation algorithms with a proven bound for this placement problem.
7. Multi-objective optimization of cost, latency and energy.
8. Larger benchmark datasets.
9. An interactive web dashboard.
10. Comparison with established optimization solvers.

CHAPTER 8 — TEAM CONTRIBUTIONS

Table 8.1: Team Contributions

Member	Contribution
Andrew Devasani	To be filled by the team
Sathyanarayana Varma	To be filled by the team
G. Daiwik Reddy	To be filled by the team
G. Keerthan Chowdary	To be filled by the team

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[2] J. Kleinberg and E. Tardos, *Algorithm Design*. Pearson, 2006.

[3] J. B. Kruskal, "On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.

[4] R. C. Prim, "Shortest Connection Networks and Some Generalizations," *Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, 1957.

[5] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[6] R. Bellman, "On a Routing Problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[7] R. W. Floyd, "Algorithm 97: Shortest Path," *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.

[8] S. Warshall, "A Theorem on Boolean Matrices," *Journal of the ACM*, vol. 9, no. 1, pp. 11–12, 1962.

[9] D. B. Johnson, "Efficient Algorithms for Shortest Paths in Sparse Networks," *Journal of the ACM*, vol. 24, no. 1, pp. 1–13, 1977.

[10] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley and Sons, 1990.

[11] Python Software Foundation, "Python Standard Library — heapq Module."

[12] SciPy Developers, "SciPy Documentation: scipy.optimize.milp."

[13] Matplotlib Development Team, “Matplotlib Documentation.”

APPENDIX

Appendix A – Sample Input Data

Network Links

Table A.1: Sample Input Data – Network Links

From	To	Latency (ms)
0	1	4
0	2	2
1	2	1
1	3	5
2	3	8
2	4	10
3	4	2
3	5	6
4	5	3

Servers and Tasks

Table A.2: Sample Input Data – Servers

Server	Capacity
0	5
1	3
2	4
3	4
4	3
5	3

Table A.3: Sample Input Data – Tasks

Task	Origin	Demand	Value
0	0	3	30
1	0	4	25
2	1	2	20
3	3	3	35
4	4	4	28
5	5	2	15

Appendix B – Placement Algorithm Workflow

The implemented system follows this sequence:

Input Data → Graph Construction → MST Backbone → Latency Matrix → Task Proposals → Knapsack Selection (repeat while tasks are pending) → Final Plan and Performance Metrics

Appendix C – Sample Output

For the sample dataset, the hybrid algorithm placed:

Placed tasks: 0, 1, 2, 3, 5 (task 4 could not be placed)

Table A.4: Sample Output

Metric	Result
Total Utility	123
Total Demand Placed	14 units
Placed Tasks	5 of 6
Average Latency	0.4 ms
Capacity Used	63.6%
Rounds Used	7

Generated Plan:

Task 0 → Server 0, Task 1 → Server 2, Task 2 → Server 1, Task 3 → Server 3, Task 5 → Server 5

The plan satisfies every server capacity, and each task is assigned to at most one server.

Appendix D – Testing Summary

The implementation was tested using automated test cases covering:

- Kruskal and Prim agreement
- Dijkstra, Bellman-Ford, Floyd-Warshall and Johnson agreement
- Negative edges (Johnson)
- Negative cycle detection
- 0/1 Knapsack against brute force
- Fractional knapsack bound
- Job sequencing
- Placement feasibility and quality against the exact optimum

Test Result: 7 tests passed successfully.

Appendix E – Project Technologies

Table A.5: Project Technologies

Component	Technology
Programming Language	Python
Graph Storage	Adjacency lists
Priority Queue	heapq (binary heap)
Exact Solver	SciPy integer programming
Charts	Matplotlib
Testing	Python test script (pytest compatible)
Data Format	JSON (results files)

Appendix F – Source Code Excerpts

F.1 0-1 knapsack by dynamic programming (src/dp.py)

```
def knapsack_01(values, weights, capacity):
    """O(n * C) time and space. Returns (best_value, chosen_indices)."""
    n = len(values)
    capacity = int(capacity)
    dp = [[0.0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        row, prev = dp[i], dp[i - 1]
        for c in range(capacity + 1):
            row[c] = prev[c]
            if w <= c and prev[c - w] + v > row[c]:
                row[c] = prev[c - w] + v
    chosen, c = [], capacity
    for i in range(n, 0, -1):
        if dp[i][c] != dp[i - 1][c]:
            chosen.append(i - 1)
            c -= weights[i - 1]
    return dp[n][capacity], chosen[::-1]
```

F.2 Greedy proposals with knapsack selection (src/placement.py)

```
def solve_greedy_dp(L, cap, d, p, o, selector="dp"):
    """Proposal rounds. Every unplaced task proposes to its best remaining server;
    each server solves a 0-1 knapsack (DP) over the proposals it received and keeps
    the best subset. Rejected tasks drop that server and re-propose. Each round every
    pending task strikes one server, so it ends in <= n_servers rounds."""
    t0 = time.perf_counter()
    n_s, n_t = len(cap), len(d)
    remaining = list(cap)
    assign = [None] * n_t
    banned = [set() for _ in range(n_t)]
    pending = set(range(n_t))
    rounds = 0
    while pending:
        rounds += 1
        proposals = {}
        for t in list(pending):
            best, bs = 0.0, None
            for s in range(n_s):
                if s in banned[t]:
                    continue
                u = _utility(L, o, p, t, s)
                if u > best:
                    best, bs = u, s
            if bs is None:
                pending.discard(t)       # no server gives positive utility
            else:
                proposals.setdefault(bs, []).append(t)
        if not proposals:
            break
        for s, ts in proposals.items():
            vals = [_utility(L, o, p, t, s) for t in ts]
            wts = [d[t] for t in ts]
            if selector == "dp":
                _, chosen = knapsack_01(vals, wts, remaining[s])
            else:   # ablation: pick proposals greedily by value density instead of DP
                chosen, room = [], remaining[s]
                for i in sorted(range(len(ts)), key=lambda i: vals[i] / wts[i], reverse=True):
                    if wts[i] <= room:
                        chosen.append(i)
                        room -= wts[i]
            keep = {ts[i] for i in chosen}
            for t in ts:
                if t in keep:
                    assign[t] = s
                    remaining[s] -= d[t]
                    pending.discard(t)
                else:
                    banned[t].add(s)
    m = _metrics(assign, L, cap, d, p, o, time.perf_counter() - t0)
    m["rounds"] = rounds
    return assign, m
```